

# Ticket Drop Platform — Requirements (v1)

Sep 24, 2026 · @Boss

## Overview

A web platform where organizers sell a fixed number of tickets at a set on-sale time. Buyers compete fairly, and the site neither oversells nor goes down.

The headline claim is correctness under a rush: **zero oversells, however many buyers arrive at once**. The resume line it should earn: *Simulated a drop of  $N$  concurrent buyers for  $M$  tickets: zero oversells, p95 time to secure a hold under  $X$  ms.*

flowchart LR

```

    A[Join queue] --> B[Wait in line]
    B --> C[Admitted]
    C --> D[Pick tickets,<br/>hold starts]
    D --> E[Pay]
    E --> F[Ticket by email<br/>and in account]
    F --> G[Scanned at door]
    D -. hold expires .-> H[Tickets back<br/>in the pool]
  
```

The buyer's path from queue to door. An expired hold returns its tickets for the next buyer.

**In v1:** general-admission ticket tiers, one on-sale time per event, a virtual queue, timed holds, payment in a provider's test mode, QR tickets by email and in the account, door check-in, and an organizer dashboard.

**Not in v1:** seat maps, resale or transfers, real money, multiple currencies, native mobile apps, dynamic pricing, waitlists after sell-out, and offline ticket scanning.

Stories are tagged Must (v1 needs it), Should (v1 if time allows) or Could (later). Story IDs like 2.2 are for commits, tests and decision records.

## Roles, terms and defaults

Four roles use the product; a fifth, the reviewer, is who the project is really for.

| Role       | Who                            | Can                                                                                     |
|------------|--------------------------------|-----------------------------------------------------------------------------------------|
| Buyer      | Anyone with a verified account | Queue, hold and buy tickets; see own tickets                                            |
| Organizer  | Creates events                 | Set up events and tiers, schedule drops, see own sales and attendees, invite door staff |
| Door staff | Invited per event              | Check in tickets for assigned events only                                               |
| Admin      | Runs the platform              | Suspend organizers or events, see platform health                                       |
| Reviewer   | A hiring manager, no account   | Try the demo, read the docs, run it locally                                             |

These terms mean the same thing everywhere in this doc.

| Term      | Meaning                                                |
|-----------|--------------------------------------------------------|
| Drop      | The on-sale moment and the rush that follows           |
| Queue     | Where buyers wait before and during a drop             |
| Admission | Letting one buyer from the queue into ticket selection |
| Hold      | Tickets reserved for one buyer for a limited time      |
| Order     | One purchase of one or more tickets                    |
| Ticket    | One entry right, shown as a signed QR code             |
| Check-in  | Door staff scanning a ticket at entry                  |

Defaults keep the acceptance criteria concrete. Change any of them, but write the change down.

| Parameter     | Default | Set by              |
|---------------|---------|---------------------|
| Hold duration | 10 min  | Organizer, 5–15 min |

| Parameter                               | Default                         | Set by    |
|-----------------------------------------|---------------------------------|-----------|
| Admission window (time to start a hold) | 5 min                           | Platform  |
| Tickets per order                       | Up to 4                         | Organizer |
| Tickets per buyer per event             | Up to 4                         | Organizer |
| Queue opens                             | 30 min before on-sale           | Platform  |
| Admission rate                          | e.g. 100 buyers per minute      | Organizer |
| Ticket email delivery                   | Within 2 min of confirmation    | Platform  |
| Door-staff invites                      | Single use, expire after 7 days | Platform  |
| Event images                            | JPEG, PNG or WebP, max 5 MB     | Platform  |
| Idle session timeout                    | 14 days                         | Platform  |
| Backups                                 | Daily, kept 7 days              | Platform  |

## 1 · Auth & authorization

Every action is checked against both role and ownership, so a guessed ID never reveals or changes someone else's data.

### 1.1 · Verified account · Must

As a buyer, I want to sign up and verify my email, so that I can join a drop.

- ☐ A buyer cannot join a queue until their email is verified.
- ☐ Verification links expire after 24 hours and work only once.
- ☐ Sign-up and password-reset responses look the same whether or not the email is registered.

### 1.2 · Sign in, sign out, reset · Must

As any user, I want to sign in, sign out and reset my password, so that my account stays mine.

- ☐ Signing out ends that session immediately.

- ☐ A reset link expires after 1 hour, works once, and signs out all other sessions when used.
- ☐ Sessions end after 14 days of inactivity.

### 1.3 · Organizers see only their own events · Must

As an organizer, I want my events, sales and attendees visible only to me, so that competitors and buyers can't see them.

- ☐ Requesting another organizer's event, sales or attendees gets the same response as a non-existent event.
- ☐ An organizer can edit or cancel only their own events.

### 1.4 · Buyers see only their own orders · Must

As a buyer, I want my orders and tickets private, so that nobody else can view or use them.

- ☐ Changing an order or ticket ID in a request never reveals another buyer's data.
- ☐ A ticket's QR code is shown only to its owner.

### 1.5 · Door staff scoped to one event · Must

As an organizer, I want to invite door staff for one event, so that I can staff the door without sharing my account.

- ☐ Door staff can check in tickets only for events they are assigned to.
- ☐ A scan shows door staff only the result, the tier and the name on the ticket, never sales or the attendee list.
- ☐ Invites are single-use and expire after 7 days.
- ☐ Access ends 24 hours after the event, or earlier if the organizer revokes it.

### 1.6 · Admin suspension · Should

As an admin, I want to suspend an organizer or an event, so that abuse stops quickly.

- ☐ A suspended event disappears from public pages, and its queue stops admitting buyers.
- ☐ Suspending never invalidates sold tickets; cancelling is a separate action that refunds every order (3.6).
- ☐ Every admin action is recorded with who did it and when.

## 1.7 · Recent sign-in for sensitive actions · Could

As an organizer, I want sensitive actions to need a recent sign-in, so that a stolen session can't do lasting damage.

- ☐ Changing email or password, or exporting attendees, requires signing in again if the last sign-in was over 15 minutes ago.

## 2 · Domain & data rules

The data rules make overselling impossible, not just unlikely. They hold under any load and any order of events.

### 2.1 · Events and ticket tiers · Must

As an organizer, I want an event with one or more ticket tiers, each with its own price and quantity, so that I control what goes on sale.

- ☐ An event has a title, description, venue, start and end time, time zone, on-sale time and image.
- ☐ Each tier has a name, price, quantity and per-order limit.
- ☐ Everyone sees event times in the event's local time zone, labeled as such.
- ☐ Once the drop starts, prices are locked and quantities can only go up.
- ☐ A quantity can never be set below the tickets already sold or held.

### 2.2 · Never oversell · Must

As an organizer, I want sales never to exceed what I put on sale, so that I never turn away a paying customer.

- ☐ For every tier, sold plus actively held tickets never exceed the quantity, at any moment, under any load.
- ☐ When two buyers try to hold the last ticket at the same instant, exactly one succeeds. The other is told none are available right now.
- ☐ Tickets from an expired hold become available to the next buyer.

### 2.3 · One clear order state · Must

As a buyer, I want my order always in one clear state, so that I know whether I have a ticket.

```
stateDiagram-v2
    [*] --> Held: tickets picked
    Held --> Paying: buyer pays
    Held --> Expired: timer runs out
    Paying --> Confirmed: provider confirms
    Paying --> Held: payment fails, time left
    Paying --> Expired: fails, no time left
    Confirmed --> Refunded: event cancelled
```

What happens when a payment confirms for an already expired order is open decision D1.

- ☐ Every order is in exactly one state from the diagram. Only the transitions shown, plus the one D1 defines, are possible.
- ☐ Orders and tickets are never deleted; every change is recorded with a time and a reason.
- ☐ No buyer is ever charged without receiving their tickets or a refund.

## 2.4 · Per-buyer limit · Must

As an organizer, I want to cap tickets per buyer, so that a few people can't buy everything.

- ☐ A buyer's held plus confirmed tickets for an event never exceed the per-buyer limit, across tabs, devices and orders.

## 2.5 · One entry per ticket · Must

As door staff, I want each ticket to admit exactly one person, so that a screenshot can't get two people in.

- ☐ Each confirmed ticket has a unique code.
- ☐ A second scan of a ticket shows "already checked in" with the time and who scanned it.
- ☐ If one ticket is scanned at two doors at the same moment, exactly one scan succeeds.
- ☐ Refunded tickets scan as invalid.

## 2.6 · Explainable numbers · Should

As an admin, I want a history of every inventory, order and payment change, so that any dashboard number can be explained.

- ☐ Sold and held counts for any tier can be rebuilt from the recorded history.

- ☐ Every notification from the payment provider is stored exactly as received, duplicates included.

### 3 · Async work

Anything slow or unreliable runs in the background, survives restarts, and can safely run twice without double effects.

#### 3.1 · Queue admission · Must

As a buyer in the queue, I want to be admitted at a steady pace, so that checkout doesn't collapse and my wait is predictable.

- ☐ Buyers are admitted at the event's configured rate, never all at once.
- ☐ The ordering rule for people who arrive before on-sale is documented and applied consistently (D2).
- ☐ An admitted buyer who doesn't start a hold within the admission window loses their turn, and the next buyer is admitted.
- ☐ When every ticket is sold, everyone still waiting is told within 10 seconds.

#### 3.2 · Hold expiry · Must

As a buyer still waiting, I want abandoned holds released quickly, so that tickets don't sit locked.

- ☐ Tickets from an expired hold return to the pool within 30 seconds of expiry.
- ☐ Expiry still happens if the server restarts in between.

#### 3.3 · Payment confirmation · Must

As a buyer, I want my order confirmed as soon as the provider confirms payment, so that I have my ticket.

- ☐ Orders are confirmed only by the provider's notification, never by the buyer's browser saying it paid.
- ☐ The same notification received twice confirms the order once and sends one email.
- ☐ Notifications arriving out of order still end in the correct final state.

### 3.4 · Stuck payments resolved · Should

As an admin, I want payments without a final answer resolved automatically, so that no order stays in limbo.

- ☐ Any order still paying after 15 minutes is checked with the provider and moved to its correct state.
- ☐ Each correction is logged and shown on the admin dashboard.

### 3.5 · Ticket delivery · Must

As a buyer, I want my tickets by email and in my account right after purchase, so that I can get in at the door.

- ☐ Tickets appear in the account as soon as the order is confirmed.
- ☐ The ticket email arrives within 2 minutes under normal conditions.
- ☐ If the email service is down, sending is retried. A final failure is flagged to admins, and the ticket stays in the account.
- ☐ Each order produces exactly one ticket email.

### 3.6 · Refunds on cancellation · Should

As a buyer, I want an automatic refund if the event is cancelled, so that I don't have to chase my money.

- ☐ Cancelling an event refunds every confirmed order and invalidates its tickets.
- ☐ Refunding continues across restarts until every order is refunded or flagged for manual review.
- ☐ Each buyer gets one cancellation email.

## 4 · Frontend state

Every screen shows current server state and has a designed loading, empty and error state. Nothing ever shows a blank page or a stale "available".

### 4.1 · Honest availability · Must

As a buyer, I want the event page to show current availability, so that I don't queue for tickets already gone.



- ☐ During a drop, availability per tier refreshes at least every 10 seconds and shows when it was last updated.
- ☐ "Sold out" appears within 10 seconds of the last ticket selling.
- ☐ While loading, no price or availability appears as a real value.
- ☐ If loading fails, the page says so and offers a retry.

#### 4.2 · Queue position · Must

As a buyer in the queue, I want to see my position and estimated wait, so that I can decide whether to stay.

- ☐ Position and estimate update at least every 10 seconds without a reload.
- ☐ Reloading or reopening the page keeps my place.
- ☐ A second tab shows the same place, never a second one.
- ☐ When admitted, I'm taken to ticket selection and shown how long my admission window lasts.

#### 4.3 · Hold countdown · Must

As a buyer holding tickets, I want a visible countdown, so that I know how long I have to pay.

- ☐ The countdown matches the server's expiry within 2 seconds, even if my device clock is wrong.
- ☐ A warning appears when 1 minute is left.
- ☐ At expiry, checkout is disabled and I'm told the tickets were released.
- ☐ Pressing Pay twice never results in two charges.

#### 4.4 · Payment result · Must

As a buyer, I want a clear result after paying, so that I know whether I have tickets.

- ☐ While confirmation is pending, the page says "Confirming payment" and updates itself.
- ☐ After 60 seconds without a result, the page says tickets will arrive by email and shows the order as pending.
- ☐ A failed payment shows the reason in plain words and allows a retry while the hold lasts.

#### 4.5 · Connection loss · Must

As a buyer on a flaky connection, I want the page to recover by itself, so that a network blip doesn't cost me my place.

- ☐ When the connection drops, a banner says so and the page reconnects automatically.
- ☐ After reconnecting, position, countdown and order state are refreshed before any action is allowed.

#### 4.6 · My tickets · Must

As a buyer, I want all my tickets in one place, so that I can show them at the door.

- ☐ With no tickets, the page says so and links to upcoming events.
- ☐ Pending orders show as pending, not missing.
- ☐ Any ticket's QR code can be opened full-screen for scanning.

#### 4.7 · Organizer live dashboard · Should

As an organizer, I want to watch my drop live, so that I can see it working.

- ☐ Shows queue length, admission rate, active holds, tickets sold and revenue, updated at least every 5 seconds.
- ☐ Before the drop, it shows a countdown and the event's settings.
- ☐ Its numbers always match the order records.

#### 4.8 · Door scanner · Must

As door staff, I want an instant, unmistakable result per scan, so that the line keeps moving.

- ☐ Results appear within 1 second on a normal connection: valid, already used, or invalid.
- ☐ Each result fills the screen and uses both color and text.
- ☐ Without a connection, the scanner says it can't verify instead of guessing.

#### 4.9 · Accessibility · Must

As a buyer using a keyboard or screen reader, I want to complete a purchase, so that the drop is fair to me too.

- ☐ The whole buyer flow works with a keyboard alone.

- ☐ Screen readers announce queue position and the 1-minute warning without moving focus or flooding the user.
- ☐ Status is never shown by color alone.
- ☐ Buyer pages pass automated WCAG 2.2 AA checks.

#### 4.10 · Speed · Should

As a buyer on a mid-range phone, I want pages to load fast, so that I'm not disadvantaged in the rush.

- ☐ Event and queue pages show their main content within 2.5 seconds on a mid-range phone over 4G.
- ☐ The queue page stays responsive while updates arrive.

### 5 · Security

Assume bots, scalpers and curious users will probe every drop. Every limit is enforced by the server, never only in the browser.

#### 5.1 · Bot and scalper limits · Must

As a buyer, I want bots slowed down, so that real people get a fair chance.

- ☐ Sign-ups, queue joins and hold attempts are rate-limited per account and per network address, with configurable limits.
- ☐ Hitting a limit shows when to retry, not an error page.
- ☐ One account holds at most one place per queue.

#### 5.2 · Unforgeable tickets · Must

As an organizer, I want tickets that can't be faked, so that only real buyers get in.

- ☐ A QR code not issued by the platform, or altered in any way, scans as invalid.
- ☐ Guessing or incrementing ticket codes never produces a valid ticket.

#### 5.3 · Trustworthy payments · Must

As the platform owner, I want only genuine payments to confirm orders, so that nobody gets free tickets.

- ☐ Notifications that can't be verified as coming from the provider are rejected and logged.
- ☐ Replaying an old genuine notification never creates a new order or ticket.
- ☐ Prices always come from the server; a price sent by the browser is ignored.
- ☐ Card details are entered only on the provider's side. The platform never receives or stores them.

#### 5.4 · Safe image uploads · Must

As the platform owner, I want uploads handled safely, so that a malicious file can't harm visitors or the server.

- ☐ Only JPEG, PNG and WebP up to 5 MB are accepted, judged by the file's content, not its name.
- ☐ Corrupt or disguised files, such as a script renamed to .jpg, are rejected with a clear message.
- ☐ Public images never carry the uploader's embedded metadata, such as GPS location.
- ☐ An uploaded file can never run as code in a visitor's browser.
- ☐ Uploads are rate-limited per organizer.

#### 5.5 · Safe attendee export · Should

As an organizer, I want to download my attendee list, so that I can use it offline.

- ☐ Only the event's organizer can export, and only that event's attendees.
- ☐ Exported values can't run as formulas when opened in a spreadsheet app.
- ☐ Every export is logged with who and when.

#### 5.6 · Validated input · Must

As the platform owner, I want every input checked on the server, so that bad data never enters the system.

- ☐ Requests with missing, malformed or out-of-range fields are rejected with an error listing each problem.
- ☐ Event names and descriptions display as plain text. They can't inject scripts or markup into pages or emails.
- ☐ Error responses never reveal internals such as stack traces or queries.

## 5.7 · Account protection · Must

As a user, I want my account protected from guessing and takeover, so that nobody can use or steal my tickets.

- ☐ After 5 failed sign-ins within 15 minutes, further attempts on that account are blocked for 15 minutes.
- ☐ Passwords found in known data breaches are rejected at sign-up and reset.

## 5.8 · Minimal personal data · Should

As a buyer, I want my personal data kept minimal and deletable, so that I stay in control of it.

- ☐ Only name, email and order history are stored for buyers.
- ☐ On account deletion, personal data is removed or anonymized, while records needed for accounting remain.

# 6 · Shipping

A reviewer can open a live URL and watch a drop happen. You can deploy, observe, roll back and restore without manual heroics.

## 6.1 · One-command local setup · Must

As a reviewer, I want to run everything locally with one command, so that I can check the project in minutes.

- ☐ From a fresh clone, one documented command starts the full system with a seeded demo event.
- ☐ Clone to running app takes under 10 minutes on a machine with only the listed prerequisites.

## 6.2 · Checked pull requests · Must

As the developer, I want every pull request checked automatically, so that broken code never reaches main.

- ☐ Every pull request runs linting, type checks, all tests and a production build.
- ☐ Any failure blocks the merge, so main is always deployable.

### 6.3 · Deploys with automatic rollback · Must

As the operator, I want merges deployed automatically and undone if they break, so that the live site stays up.

- ☐ Merging to main deploys to production with no manual steps.
- ☐ If the new version fails its health checks, the previous version returns automatically and the failure is reported.
- ☐ Rollback has been demonstrated once by shipping a deliberately broken build, with evidence in the README.

### 6.4 · Public demo drop · Should

As a reviewer, I want to start a demo drop and watch it, so that I see the hard part working in under a minute.

- ☐ Any visitor can reset and start a demo event where simulated buyers compete for the tickets.
- ☐ The visitor watches the queue drain, holds expire and the event sell out on the live dashboard.
- ☐ Demo drops are capped and rate-limited so they can't exhaust the server or the budget.
- ☐ The demo is clearly labeled, and payments run in the provider's test mode only.

### 6.5 · Observability · Must

As the operator, I want to see what the system is doing during a drop, so that I spot problems before users do.

- ☐ Logs are structured, and one identifier traces each request and background job end to end.
- ☐ A dashboard shows request and error rates, response times, queue length, active holds, job backlog and payment notifications.
- ☐ Alerts fire when the site is down, errors pass a threshold, the job backlog grows for 5 minutes, or notifications fail verification.
- ☐ Captured errors carry enough context to reproduce them, without personal data.

## 6.6 · Backups you've restored · Must

As the operator, I want automatic backups that I've actually restored, so that data loss is recoverable.

- ☐ The database is backed up at least daily, kept for 7 days, and stored apart from the production server.
- ☐ A restore into a fresh environment has been done once, with the steps and time taken documented.

## 6.7 · Secrets out of the repo · Must

As the operator, I want secrets kept out of the code, so that the repository can be public.

- ☐ No secret, key or password is ever committed, and a CI check blocks attempts.
- ☐ Every setting is documented with a safe example value.

## 6.8 · Cost ceiling · Should

As the owner, I want hosting costs capped, so that the demo can stay online for months.

- ☐ Monthly hosting stays under a budget you set, stated in the README.
- ☐ Load tests run locally or in a separate environment, never against the public demo.

# 7 · Documentation

A reviewer should grasp the project from the README's first screen, run it in under 10 minutes, and find the reasoning behind every major decision.

## 7.1 · README that sells · Must

As a reviewer, I want the README's first screen to show what this is and why it's hard, so that I decide to look deeper.

- ☐ The first screen holds a one-sentence problem statement, the live demo link, a short GIF of a drop and the headline measured result.
- ☐ Below that: an architecture diagram, tech choices with one-line reasons, how to run, and how to test.
- ☐ Known limitations and trade-offs are stated plainly.

## 7.2 · Decision records · Must

As a reviewer, I want major decisions written down, so that I can judge engineering judgment, not just code.

- ☐ Every open decision below has a record with context, options considered, the choice and its consequences.
- ☐ Records are dated and never rewritten. A changed decision gets a new record that supersedes the old one.

## 7.3 · API reference · Must

As a reviewer, I want every endpoint documented with an example, so that the API is understandable without reading code.

- ☐ Each endpoint lists inputs, outputs, errors, required role and rate limits.
- ☐ CI verifies the reference matches the running app, so it can't drift.

## 7.4 · Load test report · Must

As a reviewer, I want the measured results and how to reproduce them, so that I can trust the headline number.

- ☐ States the setup (hardware, buyers, tickets) and results (oversells, p50/p95/p99 times, error rate), with a graph of the drop over time.
- ☐ Anyone can rerun it with one documented command.
- ☐ Bottlenecks found along the way are described with before-and-after numbers.

## 7.5 · Runbook · Should

As the operator, I want steps for likely incidents, so that I can act fast under pressure.

- ☐ Covers payment provider outage, email outage, stuck background jobs, site down mid-drop, and restoring from backup.
- ☐ Each entry lists symptoms, how to confirm, what to do, and how to verify the fix.

## 7.6 · Readable history · Should

As a reviewer, I want a readable project history, so that I can see how it evolved.

- ☐ Commits are small and described, and each pull request names the story IDs it implements.



- ☐ A changelog lists user-visible changes per release.

## 8 • Tests

Tests prove the rules that matter most (no oversells, no double charges, no data leaks) instead of chasing a coverage percentage.

### 8.1 • The oversell test • Must

As a reviewer, I want proof that overselling is impossible, so that I trust the core claim.

- ☐ Fires many simultaneous hold attempts at the last few tickets, such as 200 attempts for 10 tickets, and checks exactly 10 succeed.
- ☐ Also covers holds expiring while new holds are being created.
- ☐ Runs on every pull request, and any oversell fails the build.

### 8.2 • Payment edge cases • Must

As the operator, I want every payment edge case tested, so that money and tickets always match.

- ☐ Covers duplicate, out-of-order, forged and late notifications, plus a failed payment followed by a retry.
- ☐ After each case, no buyer is charged without tickets or a refund, and no ticket exists without a payment.

### 8.3 • Access-control matrix • Must

As the operator, I want every role checked against every protected action, so that permission bugs can't slip in.

- ☐ Covers each role, including signed-out visitors, against each protected action, including other users' resources.
- ☐ A new protected action without a matrix entry fails the build.

### 8.4 • Time-based rules • Must

As the developer, I want time-dependent rules tested without real waiting, so that deadlines are verified precisely.

- ☐ Hold expiry, admission windows, link expiry and door-staff access are tested just before, at and just after each deadline.
- ☐ These tests finish in seconds.

### 8.5 · End-to-end journey · Must

As a reviewer, I want one test that walks the whole journey, so that I know the pieces fit.

- ☐ A browser test goes from sign-up through queue, hold, test payment and ticket email to a successful door scan, then a rejected second scan.
- ☐ It runs in CI against a production-like environment.

### 8.6 · Load test · Must

As a reviewer, I want a repeatable load test of a full drop, so that the headline number is reproducible.

- ☐ Simulates N buyers for M tickets, with N and M set after a first baseline run.
- ☐ Passes only with zero oversells, zero double charges, under 1% errors, and p95 hold time under your target.
- ☐ Each run's results are saved with the date and the commit tested.

### 8.7 · Frontend states · Should

As the developer, I want each screen's loading, empty, error and live states tested, so that UI regressions are caught.

- ☐ Every state required in section 4 has at least one automated test.
- ☐ Automated accessibility checks run on buyer pages in CI.

### 8.8 · Known attacks · Should

As the operator, I want known attacks tried automatically, so that defenses can't silently break.

- ☐ Covers forged and altered QR codes, disguised uploads, script injection in event text, formula injection in exports, rate-limit evasion and guessed IDs.

## Open decisions

These are yours to decide. Each is a likely interview question, so record what you chose and what you gave up (7.2).

- ☐ D1 · A payment confirms after its hold expired, and the tickets may already be gone. What happens?
- ☐ D2 · Buyers who arrive before on-sale: are they ordered by arrival, or randomly when the sale opens?
- ☐ D3 · How does a buyer keep one place across reloads and devices, and what stops one person queuing many times?
- ☐ D4 · How do live updates reach the browser, and what happens when that connection drops?
- ☐ D5 · Where is the no-oversell rule (2.2) enforced, and how do you prove it holds under concurrency?
- ☐ D6 · How are tickets made unforgeable (5.2)?
- ☐ D7 · What is the smallest architecture that meets the load target: one service or several?
- ☐ D8 · How do public demo drops (6.4) stay within your budget?

## Definition of done for v1

v1 is done when a stranger can see it work, run it, and verify the headline claim without talking to you.

- ☐ Every Must story meets its acceptance criteria.
- ☐ The live demo is up, with payments in test mode.
- ☐ The load test report shows zero oversells at your target load.
- ☐ Automatic rollback and a backup restore have each been demonstrated once.
- ☐ The README, API reference and a decision record for every open decision are published.
- ☐ CI is green on main, and every Must story links to at least one test.
- ☐ Your resume line is filled in with measured numbers.